

Programming Languages Evaluation Criteria

Programming Languages Intro

Dr. Igor Steinmacher

What makes a language “good”?

Simplicity

Reliability

Support

Abstraction

Orthogonality

Cost

Simplicity

- Before the 70's the main thing about languages was EFFICIENCY
- But... Maintenance is the most expensive phase of development
 - Maintenance requires **reading** and **understanding**



“Any fool can write code that a computer can understand. Good programmers write code that humans can understand.”

Kent Beck.



Simplicity

- Too many basic components is harder to learn
 - Developers tend to learn a subset of the language and ignore the other components
- Different ways to perform similar operations

```
cont = cont + 1  
cont++  
cont += 1  
++cont
```

Simplicity

- Blocks, identifiers and keywords

Pascal

```
i := 0;
while i < 10 do
begin
    if i mod 2 = 0 then
    begin
        writeln('EVEN');
    end
    else
    begin
        writeln('ODD');
    end;
    i := i+1;
end;
```

C

```
while (i < 10) {
    if (i%2==0){
        printf("even")
    }
    else {
        printf("odd")
    }
    i++
}
```

ADA

```
while (i < 10) loop
    if (i mod 2 = 0) then
        put_line("even");
    else
        put_line("odd");
    end if;
    i := i+1;
end loop;
```

Python

```
while i < 10:
    if i%2 == 0:
        print ('even')
    else:
        print ('odd')
    i += 1
```

Reliability

- The program behavior is the same every time it is executed with the same inputs → Determinism
- The program behavior is the same in different platforms
- The program to comply with its specifications, under all conditions.

Reliability

Type checking

Exception
handling

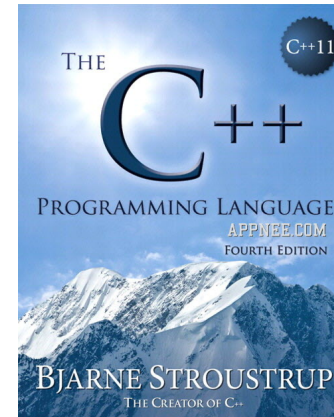
Aliasing

Avoid
memory leaks

Readability

Support

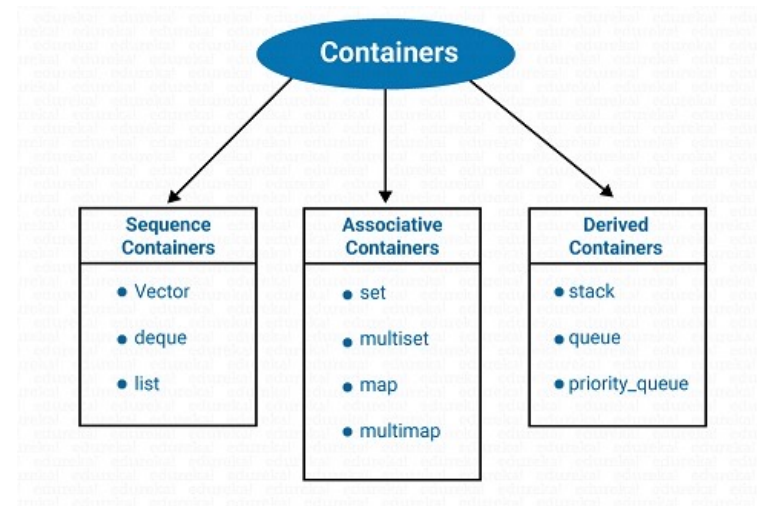
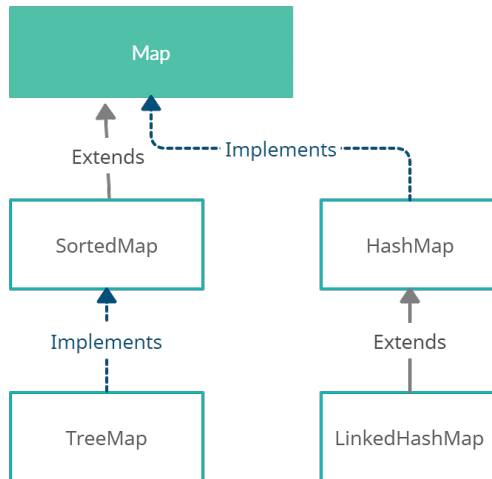
- Easy to access, install, learn → Resources!
- Points to consider:
 - Public/Open source compilers
 - Courses and tutorials
 - Book availability
 - Number of users
 - Communities



This Photo by Unknown Author is licensed under CC BY

Abstraction

- Ability to define and use complex structures or operations ignoring the complexities
- Developers can use a specific feature without understanding how it works in details



Orthogonality

- Commands and resources are built over a small set of independent primary operations
- adding two 32-bit integer values

- IBM mainframes

A Reg1, memory_cell

AR Reg1, Reg2

```
Reg1 ← contents(Reg1) + contents(memory_cell)  
Reg1 ← contents(Reg1) + contents(Reg2)
```

- VAX minicomputers

ADDL operand_1, operand_2

```
operand_2 ← contents(operand_1) + contents(operand_2)
```

The more orthogonal a language is, less rules it requires to write correct programs

Orthogonality

- However...
 - Too much orthogonality is a harm
 - Too many combinations may lead to unnecessary complexity
- In Javascript it is possible to apply the operator “+” between a string and an integer

```
3 + 4 + "5"  
7 + "5"  
"75"
```

Costs

Developer Training

Execution Time

Compilation Time

Maintenance

External tools and libraries

**See you in our next
Lesson!**